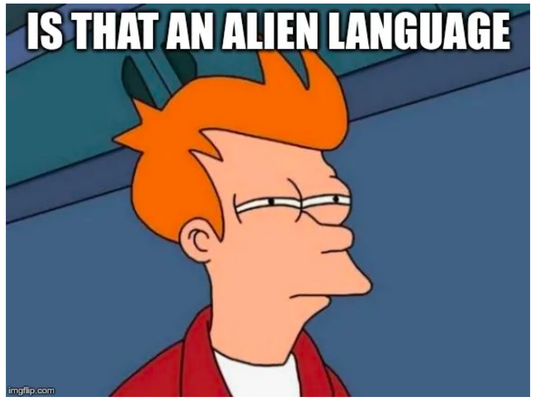
Regular Expression

- RegEx Syntax
- Examples

Kiara, S1 2024

Regular Expression



- **What is RegEx?**
  - A sequence of characters that represents a **pattern** that **matches** strings or pieces of strings

    @([a-zA-Z0–9_.+-]+)\.[a-zA-Z0–9_.+-]
- **Why use RegEx?**
  - Transferable between different programming languages (supported by Python, Java, Javascript, etc)
  - Normally used to super-fast validation, extraction, replacing,
    - **Validate** that a piece of text (or a portion of that text) matches some pattern
    - **Find fragments** of some **text** that match some pattern
    - **Extract fragments** of some **text**
    - **Replace** fragments of text with other text

Regular Expression

- **How to use RegEx?**
  - Syntax (more next page)
    - Most characters stand for themselves, e.g., match ‘hello’ by using ‘hello’
    - Certain characters (**metacharacters)**, have special meaning and must be escaped (usually with \ if you want to use them as characters. e.g., **( ) [ ] { } ^ $. \ ? * + |**
    - Examples:

```
^(|(([A-Za-z0-9]+_+)|([A-Za-z0-9]+\-+)|([A-Za-z0-9]+\.+)|([A-Za-z0-9]+\++))*[A-Za-z0-9]+@((\w+\-+)|(\w+\.))
*\w{1,63}\.[a-zA-Z]{2,6})$
```

  - Powerful Tools:
    - https://pythex.org/
    - https://regex101.com/
    - https://regexper.com/
    - https://regexr.com/
    - https://regexone.com/

Character set & OR operator : | or []

- **[]: character set ,** *e.g., [fit5196], [abcd], [John Smith]*
- **[a-z],**
  - match any 1 character in the brackets (*but only 1). *[abcd0987 =]*
  - Character ranges can be indicated by giving two characters and separating them by a ‘-‘.
  - Example:
    - match 1 single digit [0-9], [0123456789]
    - match 1 single character [a-zA-Z], [a-z0-9], [^0-9]
- **(m|n)**: or and captured

Character classes : \d \w \s and .

- **.** : match any character, except for line terminators \n
- **\d**: match a single character that is a digit, same as [0-9]
- **\D**: match a single non-digit character, same as [^0-9] or [^\d]
- **\w**: match a word character, i.e. alphanumeric character plus underscore)
- **\W**: match a non-word character, same as
- **\s**: match a whitespace character (includes tabs and line breaks). same as [\r\n\t\f\v]. \t indicates taps, \n line feeds, \r carriage returns, \f form feed, \v vertical tabs.
- **\S**: match a non-whitespace character; which is equivalent to [^ \t\n\r\f\v]

Quantifiers/Repetition: * + ? and {}

- abc*: match a string with a pattern (many times) or without the pattern *(0 or more times)*
- **+**: match a string with a pattern (once) or many time *(1 or more times),*
- **?**: match a string with a pattern (once) or without the pattern *(0 or 1 time)*
  - *? in the (?:) -> non-capturing parentheses*
- **{ }**: identify the exact number of occurrence
  - **{2}**: match **exactly m repetition** of preceding pattern,
  - **{2,}**: match **from m to more repetition** of preceding pattern,
  - **{2,4}**: match **exactly from m to n repetition** of preceding pattern,

Anchors: ^ and $

- **^**: start of input
  - match the string/text that starting with the given pattern
  - *in [], means the negated or complemented set [^0-9], [^\d]*

- **$**: end of input
  - match entire that ends with the given pattern
  - *it will filter out the partial correct ones*

Grouping and capturing : ()

- **( )**: group or capturing group
  - match whatever regular expression is inside the parentheses, and indicates the start and end of a group
  - (?:): has ‘?:’ at the beginning of the group means it is a non-capturing group, thus the pattern in the parentheses will be matched but not captured and saved as a group
  - (\d.?):

Matching Special character

Backslash or Escape-character

- ****: a sign indicate a special regex syntax, but it can also use as a **escape sign** to match special character, like those symbols have special ‘regex’ meaning
- indicate the invoking the special meaning of special characters
  - e.g., match [] & () & \, **\[**, **\(**, ****
  - **[title]**
- tell python, after the \ is not a special regex, but raw character/string

Greedy and Lazy match

- The quantifiers (* + {}) are greedy operators, so they expand the match as far as they can through the provided text.
- Use ? to make it a lazy match: match as little as possible before reaching the next regex
  - ***?** : matches the previous token between zero and unlimited times, as few times as possible
  - **+?** : matches the previous token between one and unlimited times, as few times as possible,
    - <.+?> matches any character one or more times included inside < and >, expanding as needed
  - **{m,n}?** : matches the previous token between m and n times, as few times as possible
- E.g., <.+> matches simple div in This is a simple div test. In order to catch only the div tag we can use a ? to make it lazy:
- Notice that a better solution should avoid the usage of . in favour of a more strict regex:
  - <[^<>]+> matches any character except < or > one or more times included inside < and >

Boundaries : \b and \B

- **\b :** performs a "whole words only" search
  - represents an anchor like’**^**’ (it is similar to $ and ^) matching positions where one side is a word character (like \w) and the other side is not a word character (for instance it may be the beginning of the string or a space character).
- **\B**: matches only if the pattern is fully surrounded by word characters
  - This matches all positions where \b doesn’t match

Back-references : \1

- **\1**: match the same text that was matched by the first capturing group, equivalent to *re.search(...).group(1)*
- ([abc])\1([de])\2\2 we can use \2 (\3, \4, etc.) to identify the same text that was matched by the second (third, fourth, etc.) capturing group

Useful Referenes for Regex

All materials on Moodle Page: Week 5 (pre-class, reading, demo)

Regular Expression Cookbook

https://cs.lmu.edu/~ray/notes/regex/

https://medium.com/factory-mind/regex-tutorial-a-simple-cheatsheet-by-examples-649dc1c3f285

Practical Example

1. **Please design a Regex to find the possible names of students in FIT5196 from:**
   - John $miths, @date-2009-05-11, posted on forum: content blah blah…
   - Anonymous @ 2011-02-20, posted on forum: Description of the blahblah&&…
   - Marryhasalittlesheep @ 2011-02-20, posted on forum: Description of the blahblah&&…
   - Josh O’neil @ 2011-02-20, posted on forum: Description of the blahblah&&…
   - Kiara Wang @ 2011-02-20, posted on forum: Description of the blahblah&&…

2. **Please design a Regex to match the title tag of the following html content:**
   - <a> <p>, simple div
   - <title>Model transformation/title>
   - <li><a href="/wiki/Model_transformation" title="Model transformation">Model transformation</a></li>
   - <li><a href="/wiki/Macro_(computer_science)" title="Macro (computer science)">Macro</a></li>
   - <header>Model transformation</header>

Practical Example

1. Please design a Regex to match date data: e.g. 12-9-2002, 12/03/1990,
    - *note: please distinguish months with 29/30/31 days

2. Please design a Regex to match IP address e.g. 191.0.1.8
    - *note: should be able to exclude invalid IP addresses

Recall from Lecture

Day:

(recall) `(0[1-9]|[12][0-9]|3[01])`

1,2,…9 [1-9]

01,02,..09 0[1-9]

10,11,…19 1[0-9]

[12][0-9]

20,21,… 29 2[0-9]

30,31 3[01]

Months:

(recall) `(0[1-9]|1[012])`

1,2,…9 or 01,02,..09 [1-9] or 0[1-9] => 0[1-9]

10,11,12 1[012]

Year

(recall) for 4 digits year 19XX/20XX: **(19|20)?\d\d**

19xx, 20xx \d\d

2-digits or 4-digits year: 12-9-90, 12-9-1990